Learning Objectives

- Understand how assembly instructions are executed on the ULM architecture
- Use the tools generated from an ISA description with ulm-generator
- Write, assemble, debug, and run simple assembly programs
- Use labels, jumps, and registers to implement basic control flow
- Recognize recurring patterns in assembly code and automate their generation
- Take first steps toward building the code generator for our compiler project

Preparation

From the preparation videos, you already know how to use the ULM Generator to build a virtual machine, debugger, assembler, and documentation from an instruction-set description. Everything required should have been installed in Session 00, so the following should work without any further setup.

In the videos, we used `ulm-ice40.isa`, which describes the instruction set implemented by ULM-on-iCE on the iCE40 FPGA. In this worksheet, we will use a slightly extended version called `hpc0.isa`.

The main difference is that `hpc0.isa` provides a few additional instructions that will make assembly programming more convenient. In particular, it provides 64-bit multiplication and division. These operations are deliberately not part of `ulm-ice40.isa`: on ULM-on-iCE, every instruction takes exactly one clock cycle, whereas a direct implementation of 64-bit multiplication or division would require more work.

Later, we will see that such operations could instead be implemented using function calls and simpler instructions — or by extending the hardware with operations that take more than one clock cycle. For now, `hpc0.isa` allows us to concentrate on assembly programming without worrying about these details.

Fetch the ISA description:

- Fetch the ISA description:
  `ulm-generator --fetch hpc0.isa`
- Build and install the virtual machine, debugger, and assembler:
  `sudo ulm-generator --install hpc0.isa`
- Generate the instruction-set documentation (`./hpc0/refman.pdf`):
  `ulm-generator --refman hpc0.isa`

Step 1 — Getting Familiar with the Generated Tools

Let us start with a very simple assembly program. Create a file `ex1.s` containing:

```
    getc    %1
    putc    %1
    halt    0
```

Note that each instruction should be indented by at least one whitespace character.
The program reads one character from the keyboard buffer into register `%1`, writes the character stored in `%1` to the output device, and finally terminates with exit code `0`.

Tasks

1. Assemble the program: `ulmas -o ex1 ex1.s`. Inspect the generated output file `ex1`. It should contain something like:

```
#TEXT 4
0x0000000000000000: 32 10 00 00 #     getc %1
0x0000000000000004: 30 10 00 00 #     putc %1
0x0000000000000008: 01 00 00 00 #     halt 0
#DATA 1
#BSS 1 0
#SYMTAB
```

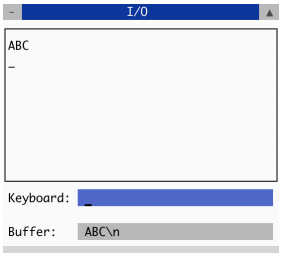
   Compare the machine-code bytes with the assembly instructions. Keep in mind what you saw in ULM on Paper: these are the bytes that eventually end up in memory and are fetched and executed by the processor.

2. Load the program into the ULM debugger: `udb-tui ex1`

   Familiarize yourself with the displays for memory and registers. The first 12 bytes of memory should contain the machine code of our three instructions.

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
0x0000000000000000	32	10	00	00	30	10	00	00	01	00	00	00	00	00	00	00

   Before starting the program, fill the keyboard buffer. In the I/O device, enter ABC followed by Return into the Keyboardfield:



   Now execute exactly one instruction using `Control-S`. Observe what happened:
   - The instruction pointer `%IP` has advanced by 4 bytes.
   - The instruction register `%IR` contains the machine instruction for `getc %1`.
   - The character `A` has been removed from the keyboard buffer.
   - Register `%1` contains the ASCII code of `A`, i.e. `0x41`.

   Continue executing the program one instruction at a time until it terminates. For each instruction, observe what changed and what did not change.
3. Quit the debugger and start it again. This time, execute the program **without filling the keyboard buffer first**. Observe what happens when `getc` is executed.
4. Finally, run the program directly from the terminal: `ulm ex1`
   Compare this experience with running the same program in the debugger.

Step 2 — Jumps and Labels

Extend `ex1.s` by an unconditional jump:

```
    getc    %1
    putc    %1
    jmp     -8
    halt    0
```

After writing a character, `jmp -8` transfers control to the instruction located 8 bytes earlier — the `getc %1` instruction.

Tasks

1. Assemble the program again and run it with `ulm ex1`. Enter some text line by line and observe what happens. The halt instruction can no longer be reached. Use `Control-C` to terminate the program.
2. Inspect the generated executable file `ex1`. You should find:

```
#TEXT 4
0x0000000000000000: 32 10 00 00 #     getc %1
0x0000000000000004: 30 10 00 00 #     putc %1
0x0000000000000008: 05 FF FF FE #     jmp -8
0x000000000000000C: 01 00 00 00 #     halt 0
#DATA 1
#BSS 1 0
#SYMTAB
```

   The instruction `jmp -8` is encoded as `05 FF FF FE`. The value `-8` itself is represented by the 24-bit pattern `FF FF FE`, corresponding to the signed value `-2`.
   Why does the assembler divide the jump offset by 4 before encoding it into the instruction?
3. **Labels:** Writing `jmp -8` works, but it forces us to calculate addresses and offsets ourselves. Assembly languages provide labels so that the assembler can do this work for us:

```
G   getc    %1
    putc    %1
    jmp     G
    halt    0
```

Assemble the program again and verify that the generated machine code is identical to the previous version. Inspect the symbol table in `ex1`. You should now find an entry for the symbol `G`. What value does it have, and why?

4. **Conditional Jumps:** Our program should terminate when `getc` reads `EOF` (End of File). In a terminal, you can produce `EOF` using `Control-D`. On the `ulm-ice40` architecture, `EOF` is represented by `0xFF`. Modify the program:

```
G   getc    %1
    subq    0xFF,   %1,     %0
    jz      H
    putc    %1
    jmp     G
H   halt    0
```

The `subq` instruction sets the zero flag if register `%1` contains `0xFF`. In this case, `jz` (jump zero) transfers control to `H`. Assemble and run the program. Enter some text and finally terminate the input using `Control-D`.

Step 3 — Reading a Decimal Integer

Let us write a slightly more interesting assembly program. It should read a non-negative integer in decimal notation and terminate with the value of that integer as its exit code.
Create `ex2.s` with:

```
 1      loadz   0,      %2
 2
 3 G    getc    %1
 4      subq    0xFF,   %1,     %0
 5      jz      H
 6      subq    '9',    %1,     %0
 7      ja      H
 8      subq    '0',    %1,     %0
 9      jb      H
10      mulq    10,     %2,     %2
11      subq    '0',    %1,     %1
12      addq    %1,     %2,     %2
13      jmp     G
14
15 H    halt    %2
```

Tasks

1. Before worrying about the details, let us see whether the program actually works. Assemble and run the program. Enter `123` followed by `Return`. After the program terminates, verify with `echo $?` that its exit code is indeed `123`.
2. Run the program again and enter a value larger than `255`, for example `1234`. You will find that the exit code is `210`. Under Unix, only the lowest 8 bits are used for the exit status. Hence, you get $1234 \bmod 2^8 = 1234 \bmod 256 = 210$.
3. **Understanding the Program:** Now let us take a closer look at what the program does.
   - `ja H` *(jump above)* performs the jump if the subtraction before it determined that **'9'** is above the value in **%1**.
   - `jb H` *(jump below)* performs the jump if `'0'` is below the value in `%1`.

   Work through the program for the input `123` and keep track of the values in `%1` and `%2`. What is the purpose of register `%2`? Explain how the instructions

```
10      mulq    10,     %2,     %2
11      subq    '0',    %1,     %1
12      addq    %1,     %2,     %2
```

   convert a sequence of decimal digits into an integer.

4. **Comparing Assembly and ABC:** The same program could be written in ABC as follows

```
@ <stdio.hdr>

fn main(): unsigned
{
    local val: int = 0;
    local ch: int;

label G:
    ch = getchar();
    if (ch == EOF) { goto H; }
    if (ch < '0') { goto H; }
    if (ch > '9') { goto H; }
    val *= 10;
    val += ch - '0';
    goto G;
label H:
    return val;
}
```

```
@ <stdio.hdr>

fn main(): int
{
    local val: int = 0;
    local ch: int;

    while ((ch = getchar()) != EOF) {
        if (ch < '0' || ch > '9') {
            break;
        }
        val = val * 10 + ch - '0';
    }
    return val;
}
```

The first ABC version resembles the assembly program much more closely. The second version expresses what we actually want to compute much more clearly. A compiler somehow has to get us from the latter to something resembling the former.

Step 4 — Simple Functions (Leaf Functions)

The following program `ex3.s` implements two simple functions `foo` (lines 8–13) and `bar` (lines 15–20), which print the strings `foo` and `bar`, respectively:

```
 1      call    foo,    %1
 2      call    bar,    %1
 3      call    foo,    %1
 4      call    bar,    %1
 5
 6      halt    0
 7
 8  foo:
 9      putc    'f'
10      putc    'o'
11      putc    'o'
12      putc    '\n'
13      ret     %1
14
15  bar:
16      putc    'b'
17      putc    'a'
18      putc    'r'
19      putc    '\n'
20      ret     %1
```

Tasks

1. Assemble the program as `ex3` and run it with `ulm ex3`. It should produce:

```
foo
bar
foo
bar
```

2. Run the program in the debugger and step through it instruction by instruction. Observe carefully what happens when

```
call    foo,    %1
```

is executed:

- execution continues at the first instruction of `foo`;
- the address of the instruction following the call is stored in register `%1`.

Continue until

```
ret     %1
```

is executed. Verify that execution continues at exactly the return address stored in `%1`.

Step 5 — What About Non-Leaf Functions?

So far, our functions have not called other functions. Let us change that in `ex4.s`. Suppose `foo` calls `bar`:

```
        call    foo,    %1
        halt    0

foo:
        putc    'f'
        putc    'o'
        putc    'o'
        putc    '\n'

        call    bar,    %1

        ret     %1

bar:
        putc    'b'
        putc    'a'
        putc    'r'
        putc    '\n'

        ret     %1
```

Tasks

Assemble and run the program. You will find that it does not work as intended.
Use the debugger to find out what goes wrong. In particular, keep an eye on register `%1`:

1. What value does `%1` contain immediately after the call to `foo`?
2. What happens to this value when `foo` calls `bar`?
3. Where does `bar` return to?
4. What happens when `foo` subsequently executes `ret %1`?

Explain to your neighbor why this approach cannot work in general.

Note: A function that calls another function is called a **non-leaf function**.

Step 6 — Using a Stack for Function Calls

We have found the problem: before foo can call bar, it somehow has to save its own return address. A stack gives us exactly what we need.

In the following program `ex5.s`, register `%1` is used as the stack pointer. Initially, it contains `0`:

```
        loadz   0,      %1
        call    foo,    %2
        halt    0
foo:
        subq    8,      %1,     %1
        movq    %2,     (%1)

        putc    'f'
        putc    'o'
        putc    'o'
        putc    '\n'
        call    bar,    %2

        movq    (%1),   %2
        addq    8,      %1,     %1
        ret     %2
bar:
        subq    8,      %1,     %1
        movq    %2,     (%1)

        putc    'b'
        putc    'a'
        putc    'r'
        putc    '\n'

        movq    (%1),   %2
        addq    8,      %1,     %1
        ret     %2
```

Tasks

1. Assemble and run the program. Verify that foo can now call bar and both functions return correctly.
2. Use the debugger to understand what happens at the beginning of each function:

```
subq    8,      %1,     %1
movq    %2,     (%1)
```

   `movq %2, (%1)` is a **store operation**: it copies the 8-byte value in register `%2` to the memory address stored in `%1`.

   Draw the stack after entering `foo` and again after entering `bar`. For each stored value, identify which return address it represents.

3. Now investigate the instructions at the end of each function:

```
movq    (%1),   %2
addq    8,      %1,     %1
```

   `movq (%1), %2` is a **fetch operation**: it copies 8 bytes from the memory address stored in `%1` into register `%2`.

   Use the debugger to verify that the correct return address is restored before each `ret`.

4. **Recursion:** Now test whether our mechanism also works when a function calls itself.

   Write a program `ex6.s` with a function `foo` that prints `foo\n` and then:

   - returns if register `%3` contains `0`;
   - otherwise decrements `%3` by one and calls itself recursively.

   Before the first call to `foo`, initialize `%3` with `42`.

Looking Ahead: Code Generator

Our next goal is to build a **code generator**: a module that generates assembly code at a higher level of abstraction. Details such as which register is available or the exact name and syntax of an instruction can then be hidden behind a small interface. Instead of choosing a register and writing the instruction ourselves, we would like to say:

Load the constant 42 into a free register.

The same interface can also hide **platform-dependent code generation**. At the end of the course, you will receive a second implementation that generates **LLVM IR** — an assembly-like intermediate representation that can be translated into native machine code for different target architectures.

There is a catch: writing the code generator itself involves quite a bit of typing. Much of it will consist of functions with several `printf` statements that emit assembly instructions. The individual pieces are usually not particularly complicated — once you have figured out how to escape `%` in a `printf` format string ;-) — there are just quite a few of them.

Therefore, we will start with a tiny subset that generates a very simple assembly program:

```
.text
loadz   0,      %1
loadz   42,     %4
loadz   13,     %5
subq    %5,     %4,     %4
loadz   123,    %4
halt    0
```

Admittedly, not a particularly useful program. ;-) The interesting part is how we generate it: we want to request *“load the constant 42 into a free register”* without specifying `%4` or the exact assembly instruction ourselves. From there, we can gradually extend the code-generator interface as we need more features.

Step 7 — First Steps with Our Code Generator

Create a subdirectory `codegen` in your session directory. The code we develop there will later become part of our compiler project. The files `xtest_gen_01.abc`, `gen.hdr`, and a skeleton of `gen.abc` are also provided for download. You may use them if necessary, but there is some value in typing the small examples yourself: it forces you to look at the interface and the code more carefully than simply copying the files.

First, create `gen.hdr`:

```
extern fn genInit();
extern fn genExit();

type GenReg: unsigned;
extern fn genReleaseReg(reg: GenReg);
extern fn genConstant(val: ->readonly char): GenReg;

enum GenInstr
{
    GEN_SUB,
};

extern fn genInstr(instr: GenInstr, left: GenReg, right: GenReg): GenReg;
```

Next, create `xtest_gen_01.abc`:

```
@ <stdio.hdr>
@ "gen.hdr"

fn main()
{
    genInit();
    local a: GenReg = genConstant("42");
    local b: GenReg = genConstant("13");
    genReleaseReg(a);
    local c: GenReg = genInstr(GEN_SUB, a, b);
    genReleaseReg(b);
    genReleaseReg(c);
    genConstant("123");
    genExit();
}
```

For now, we only have the interface of our code generator — `gen.abc` does not exist yet. Let us first understand what we want this interface to do:

- `genInit()` generates the code that, according to our conventions, must always appear at the beginning of a program. For now, this includes initializing the stack pointer. Later, it will also generate code that calls our main function.
- `genExit()` generates the code required at the end of our program.
- `GenReg` is simply an alias for unsigned. The alias makes its purpose explicit in our interface: a value of type GenReg is used as an identifier for a register.
- `genConstant("42")` generates code that loads the constant `42` into a free register and returns a `GenReg` telling us which register was chosen.
- `genReleaseReg(a)` tells the code generator that this register is no longer needed and may be reused.
- `genInstr(GEN_SUB, a, b)` generates code that subtracts the **right operand** `b` from the **left operand** `a` (i.e., computes `a - b`). The result is stored in a free register, which is returned as a `GenReg`.

Before continuing, go through `xtest_gen_01.abc` line by line and keep track of which registers are currently in use and which are free.

Behind the Interface

Our public interface in `gen.hdr` deliberately contains only what a user of the code generator needs. The implementation in `gen.abc`, however, also needs some internal machinery.

For example, the public interface provides `genReleaseReg()` to release a register, but no function for obtaining a free one. Internally, `gen.abc` provides `getReg()` for exactly this purpose. It also maintains the information required to keep track of which registers are currently in use.

There is another internal helper, `setSegment()`. Our generated assembly code may contain different segments such as `.text`, `.data`, and `.bss`. This function keeps track of the current segment and emits a segment directive when necessary.

None of these implementation details need to be visible in `gen.hdr`. They are part of how our code generator works, not how it is used.

Create `gen.abc` from the following skeleton:

```
@ <stdio.hdr>
@ "gen.hdr"

//-----------------------------------------------------------------------------

enum Segment
{
    UNDEFINED,
    TEXT,
    DATA,
    BSS,
};

global curSegment: Segment  = UNDEFINED;

fn setSegment(seg: Segment)
{
    assert(0 && "TODO");
}

//-----------------------------------------------------------------------------

global usedReg: array[16] of int;
global numUsedReg: int;

fn getReg(): GenReg
{
    assert(0 && "TODO");
    return 0;
}

//-----------------------------------------------------------------------------

fn genReleaseReg(reg: GenReg)
{
    assert(0 && "TODO");
}

fn genInit()
{
    setSegment(TEXT);
    printf("\tloadz \t0, \t%%1\n");
}

fn genExit()
{
    printf("\thalt \t0\n");
}

fn genConstant(val: ->readonly char): GenReg
{
    assert(0 && "TODO");
    return 0;
}

//-----------------------------------------------------------------------------

fn genInstr(instr: GenInstr, left: GenReg, right: GenReg): GenReg
{
    assert(0 && "TODO");
    return 0;
}
```

Tasks

1. Compile the test program together with the provided stub implementation:

   ```
   abc -o xtest_gen_01 xtest_gen_01.abc gen_stubs.abc
   ```

   Run the resulting program. You should get something like:

   ```
   gen_stubs.abc:18: Assertion '0 && "TODO"' failed.
   Abort trap: 6
   ```

   The assertion is triggered in `setSegment()`.
2. Replace the stub of `setSegment()` with:

   ```
   fn setSegment(seg: Segment)
   {
       if (seg == curSegment) {
           return;
       } else {
           switch (seg) {
               default: assert(0 && "not all cases handled");
               case TEXT: printf("\t.text\n"); break;
               case DATA: printf("\t.data\n"); break;
               case BSS: printf("\t.bss\n"); break;
           }
           curSegment = seg;
       }
   }
   ```

   `curSegment` is a global variable used internally by the code generator to keep track of the current segment. Assembly code is emitted only when the segment actually changes.
   From now on, after replacing a stub, compile and run the test program again. Follow the execution until you reach the next `TODO` — that tells you where to continue.
3. Replace the stub for `genConstant()` with:

   ```
   fn genConstant(val: ->readonly char): GenReg
   {
       local reg: GenReg = getReg();
       setSegment(TEXT);
       printf("\tloadz \t%s, \t%%%d\n", val, reg);
       return reg;
   }
   ```

   The function first obtains a free register, makes sure that we are generating code in the `.text` segment, emits the corresponding load instruction, and finally returns the register that was used. Take a closer look at the format string: `"\tload \t%s, \t%%%d\n"`. Here, `\t` generates a tab character. To print a `%` with `printf`, we have to write `%%`. Hence, `%%%d` prints a literal `%` followed by the register number. For example, if `reg == 4`, it produces `%4`.
4. The next missing piece is `getReg()`. We use a deliberately simple mechanism to keep track of free registers:

   ```
   fn getReg(): GenReg
   {
       for (local r: GenReg = 4; r < sizeof(usedReg)/sizeof(usedReg[0]); ++r) {
           if (!usedReg[r]) {
               usedReg[r] = 1;
               ++numUsedReg;
               return r;
           }
       }
       assert(0 && "getReg: out of registers!\n");
       return 0;
   }
   ```

   Register `%0` is our zero register, and `%1` is already reserved as the stack pointer. Registers `%2` (currently used for the return address) and `%3` will also acquire special roles later. We therefore anticipate this and use only registers %4 and above for general-purpose operations.

   The mechanism itself is intentionally simple: `usedReg[r]` tells us whether register `r` is currently in use. `getReg()` searches from `%4` upwards, marks the first free register as used, and returns its number. `numUsedReg` keeps track of how many general-purpose registers are currently in use.

5. Following the TODOs brings us to `genReleaseReg()`. Replace its stub with:

```
fn genReleaseReg(reg: GenReg)
{
    assert(usedReg[reg]);
    usedReg[reg] = 0;
    --numUsedReg;
}
```

   The function marks the register as free and updates the number of registers currently in use. The assert catches an attempt to release a register that is already free.

6. The final TODO brings us to `genInstr()`. Replace its stub with[1]:

```
fn genInstr(instr: GenInstr, left: GenReg, right: GenReg): GenReg
{
    local dest: GenReg = getReg();
    setSegment(TEXT);
    switch (instr) {
        case GEN_SUB:
            printf("\tsubq \t%%%d, \t%%%d, \t%%%d\n", right, left, dest);
            return dest;
        default: assert(0 && "not all cases handled");
    }
    assert(0 && "ups: internal error (forgotten return in a case??)");
    return 0;
}
```

   The `switch` gives us a simple pattern for extending the code generator later: each `GenInstr` is translated into the corresponding assembly instruction. The result is written to a free register obtained by `getReg()`, and its `GenReg` is returned.

7. Our first tiny code generator is complete. Now let us check whether the generated assembly code actually works. Redirect the output of the test program to a file:

   `./xtest_gen_01 > foo.s`

   Take a quick look at foo.s and compare it with the assembly code shown in **Looking Ahead**. Now assemble the generated code: `ulmas -o foo foo.s`

   Finally, load the program into the debugger: `udb-tui foo` and execute the program instruction by instruction.

Step 8 — Extending the Code Generator

Our code generator currently knows only one binary instruction: `GEN_SUB`. Let us add another one. Create a new test program `xtest_gen_02.abc`:

```
@ <stdio.hdr>
@ "gen.hdr"

fn main()
{
    genInit();
    local a: GenReg = genConstant("42");
    local b: GenReg = genConstant("13");
    genReleaseReg(a);
    local c: GenReg = genInstr(GEN_SUB, a, b);
    local d: GenReg = genInstr(GEN_ADD, a, b);
    genReleaseReg(b);
    genReleaseReg(c);
    genReleaseReg(d);
    genExit();
}
```

Extend the code generator so that this program produces:

```
        .text
        loadz  0,     %1
        loadz  42,    %4
        loadz  13,    %5
        subq   %5,    %4,    %4
        addq   %5,    %4,    %6
        halt   0
```

[1] In our ULM assembly syntax, `subq %1, %2, %3` subtracts the value in `%1` from the value in `%2` and stores the result in `%3`. In other words, `%3 = %2 - %1`. So source operands appear in the reverse order of the arithmetic expression.